

RAG_Assignment

March 30, 2026

1 RAG with HuggingFace and Milvus - Student Notebook

In this assignment, you will implement a complete RAG (Retrieval-Augmented Generation) pipeline using: - **Dataset:** HuggingFace Documentation (`m-ric/huggingface_doc`) - **Vector Store:** Milvus - **Embeddings:** BGE-small-en-v1.5 - **LLM:** Microsoft Phi-3-mini-4k-instruct/"Qwen/Qwen2-1.5B-Instruct" - **Evaluation:** Opik (AnswerRelevance, Hallucination)

1.1 Instructions

1. Read through each section carefully
2. Complete the code in cells marked with # TODO
3. Run all cells in order
4. Verify your implementation with the evaluation cells

[]:

```
##https://github.com/milvus-io/milvus
```

1.2 1. Setup

Install required dependencies and configure environment.

```
[7]: from sympy.codegen.fnodes import dimension
      # Install dependencies
      !pip3 install -q pymilvus sentence-transformers datasets transformers torch
      ↪accelerate opik tqdm
```

[notice] A new release of pip is

available: 25.2 -> 26.0.1

[notice] To update, run:

```
pip3 install --upgrade pip
```

```
[8]: import os
      import json
      from typing import List, Dict, Tuple
```

```
[9]: # Set your HuggingFace token for model access
# You can get one at: https://huggingface.co/settings/tokens
os.environ["HF_TOKEN"] = "hf_XXXX" # Replace with your token

# Opik configuration (optional - for generation evaluation)
# Get your API key at: https://www.comet.com/
os.environ["OPIK_API_KEY"] = "xxxx" # Replace with your Opik API key if
    ↪available

print("Environment configured!")
```

Environment configured!

1.3 2. Data Loading

Load the HuggingFace documentation dataset.

```
[10]: from datasets import load_dataset

# Load the HuggingFace documentation dataset
dataset = load_dataset("m-ric/huggingface_doc", split="train")

print(f"Dataset loaded with {len(dataset)} documents")
print(f"Columns: {dataset.column_names}")
print(f"\nSample document (first 500 chars):")
print(dataset[0]["text"][:500])
```

/Users/kishankunal/workspace/AppliedAI/.venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: <https://github.com/urllib3/urllib3/issues/3020>

```
warnings.warn(
/Users/kishankunal/workspace/AppliedAI/.venv/lib/python3.9/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
Generating train split: 100%|      | 2647/2647 [00:00<00:00, 18103.98
examples/s]
```

Dataset loaded with 2647 documents
Columns: ['text', 'source']

Sample document (first 500 chars):
Create an Endpoint

After your first login, you will be directed to the [Endpoint creation page](<https://ui.endpoints.huggingface.co/new>). As an example, this guide will go through the steps to deploy [distilbert-base-uncased-finetuned-

sst-2-english](https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-english) for text classification.

1. Enter the Hugging Face Repository ID and your desired endpoint name:

= len(text):
```

```
 break
```

```
 start += step
```

```
return chunks
```

```
def chunk_all_documents(documents: List[Dict], chunk_size: int = 1000,
```

```
 chunk_overlap: int = 200) -> List[Dict]:
```

```
 """
```

*Chunk all documents and preserve metadata.*

*Args:*

*documents: List of document dicts with 'text' and 'source' keys*

*chunk\_size: Maximum chunk size*

```

 chunk_overlap: Overlap between chunks

 Returns:
 List of chunk dicts with 'text', 'source', and 'chunk_id' keys
 """
 all_chunks = []
 chunk_id_counter = 0

 for doc in documents:
 text = doc.get("text", '')
 source = doc.get("source", 'Unknown')

 #split the current document

 individual_chunks = chunk_document(text, chunk_size, chunk_overlap)

 #Re-wrap with metadata
 for content in individual_chunks:
 all_chunks.append({
 'chunk_id': chunk_id_counter,
 'text': content,
 'source': source,
 })
 chunk_id_counter += 1

 return all_chunks

```

```

[14]: # Test your chunking implementation
test_text = "A" * 2500 # 2500 characters
test_chunks = chunk_document(test_text, chunk_size=1000, chunk_overlap=200)

print(f"Test: 2500 char text with chunk_size=1000, overlap=200")
print(f"Expected chunks: ~4")
print(f"Your chunks: {len(test_chunks)}")

if len(test_chunks) >= 3 and len(test_chunks) <= 5:
 print(" Chunking test passed!")
else:
 print(" Check your chunking implementation")

```

```

Test: 2500 char text with chunk_size=1000, overlap=200
Expected chunks: ~4
Your chunks: 3
Chunking test passed!

```

```

[15]: # Create chunks from all documents
CHUNK_SIZE = 1000
CHUNK_OVERLAP = 200

```

```

chunks = chunk_all_documents(documents, CHUNK_SIZE, CHUNK_OVERLAP)

print(f"\nCreated {len(chunks)} chunks from {len(documents)} documents")
print(f"Average chunks per document: {len(chunks) / len(documents):.2f}")

Show sample chunk
if chunks:
 print(f"\nSample chunk:")
 print(f" ID: {chunks[0]['chunk_id']}")
 print(f" Source: {chunks[0]['source']}")
 print(f" Text (first 200 chars): {chunks[0]['text'][:200]}...")

```

Created 5535 chunks from 500 documents  
Average chunks per document: 11.07

Sample chunk:

```

ID: 0
Source: huggingface/hf-endpoints-
documentation/blob/main/docs/source/guides/create_endpoint.mdx
Text (first 200 chars): Create an Endpoint

```

After your first login, you will be directed to the [Endpoint creation page](https://ui.endpoints.huggingface.co/new). As an example, this guide will go through the steps to deploy...

## 1.5 4. Embeddings

Generate vector embeddings for each chunk using BGE-small-en-v1.5.

### 1.5.1 Your Task

Implement the `generate_embeddings` function that: 1. Processes texts in batches for memory efficiency 2. Uses the SentenceTransformer model to generate embeddings 3. Returns embeddings as a list of lists (for Milvus compatibility)

### 1.5.2 Hints

- Use `model.encode()` with `normalize_embeddings=True` for cosine similarity
- Process in batches to avoid memory issues
- Convert numpy arrays to lists using `.tolist()`

```

[16]: from sentence_transformers import SentenceTransformer

Load the embedding model
EMBEDDING_MODEL = "BAAI/bge-small-en-v1.5" #Use any model of your choice from
↳Sentence Transformers
embedding_model = SentenceTransformer(EMBEDDING_MODEL)

```

```

print(f"Loaded embedding model: {EMBEDDING_MODEL}")

Test embedding
test_embedding = embedding_model.encode(["This is a test"],
 ↪normalize_embeddings=True)
EMBEDDING_DIM = len(test_embedding[0])
print(f"Embedding dimension: {EMBEDDING_DIM}")

```

Loaded embedding model: BAAI/bge-small-en-v1.5  
 Embedding dimension: 384

```

[17]: # =====
TODO: IMPLEMENT EMBEDDING GENERATION (15 points)
=====

def generate_embeddings(texts: List[str], model: SentenceTransformer,
 ↪batch_size: int = 32) -> List[List[float]]:
 """
 Generate embeddings for a list of texts.

 Args:
 texts: List of text strings to embed
 model: SentenceTransformer model
 batch_size: Number of texts to process at once

 Returns:
 List of embedding vectors (as lists of floats)
 """
 all_embeddings = []

 for i in range(0, len(texts), batch_size):
 batch_texts = texts[i : i + batch_size]
 batch_embeddings = model.encode(
 batch_texts,
 normalize_embeddings=True,
 show_progress_bar=False
)
 all_embeddings.extend(batch_embeddings.tolist())

 return all_embeddings

```

```

[18]: # Test your embedding generation
test_texts = ["Hello world", "This is a test", "RAG is cool"]
test_embeddings = generate_embeddings(test_texts, embedding_model)

print(f"Generated {len(test_embeddings)} embeddings")

```

```

print(f"Embedding dimension: {len(test_embeddings[0]) if test_embeddings else 0}")

if len(test_embeddings) == 3 and len(test_embeddings[0]) == 384:
 print(" Embedding generation test passed!")
else:
 print(" Check your embedding implementation")

```

Generated 3 embeddings  
Embedding dimension: 384  
Embedding generation test passed!

```

[19]: # Generate embeddings for all chunks
chunk_texts = [chunk["text"] for chunk in chunks]
embeddings = generate_embeddings(chunk_texts, embedding_model)

print(f"\nGenerated {len(embeddings)} embeddings")
if embeddings:
 print(f"Embedding dimension: {len(embeddings[0])}")
 print(f"Sample embedding (first 10 values): {embeddings[0][:10]}")

```

Generated 5535 embeddings  
Embedding dimension: 384  
Sample embedding (first 10 values): [-0.07532957941293716, -0.02750801295042038, -0.039956167340278625, -0.040492139756679535, 0.03334002196788788, 0.0429651252925396, -0.043336283415555954, -0.04493828862905502, -0.055543188005685806, 0.026720302179455757]

## 1.6 5. Vector Store (Milvus)

Store embeddings in Milvus for efficient similarity search.

### 1.6.1 Your Task

1. Implement `setup_milvus_collection` to create a new collection
2. Implement `insert_data_to_milvus` to insert chunks and embeddings

### 1.6.2 Hints

- Use `client.has_collection()` to check if collection exists
- Use `client.drop_collection()` to remove existing collection
- Use `client.create_collection()` with dimension and metric\_type parameters
- Use `client.insert()` to add data

```

[20]: import setuptools
import milvus_lite
import pkg_resources

```

```
print(f"setuptools version: {pkg_resources.get_distribution('setuptools').
 ↪version}")
print("Success! Libraries are recognized.")
```

setuptools version: 81.0.0

Success! Libraries are recognized.

/Users/kishankunal/workspace/AppliedAI/.venv/lib/python3.9/site-packages/milvus\_lite/\_\_init\_\_.py:15: UserWarning: pkg\_resources is deprecated as an API. See [https://setuptools.pypa.io/en/latest/pkg\\_resources.html](https://setuptools.pypa.io/en/latest/pkg_resources.html). The pkg\_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.

```
from pkg_resources import DistributionNotFound, get_distribution
```

```
[21]: from pymilvus import MilvusClient

Initialize Milvus client (uses Milvus Lite - stores data locally)
MILVUS_DB_PATH = "./hf_docs_milvus.db"
milvus_client = MilvusClient(uri=MILVUS_DB_PATH)

COLLECTION_NAME = "hf_documentation"

print(f"Milvus client initialized with database: {MILVUS_DB_PATH}")
```

Milvus client initialized with database: ./hf\_docs\_milvus.db

```
[22]: # =====
TODO: IMPLEMENT MILVUS COLLECTION SETUP (10 points)
=====

def setup_milvus_collection(client: MilvusClient, collection_name: str, ↵
 ↪embedding_dim: int):
 """
 Create a Milvus collection for storing document embeddings.

 Args:
 client: MilvusClient instance
 collection_name: Name of the collection to create
 embedding_dim: Dimension of the embedding vectors
 """
 if client.has_collection(collection_name):
 client.drop_collection(collection_name)
 print(f"Dropped collection {collection_name}")

 client.create_collection(
 collection_name=collection_name,
 dimension=embedding_dim,
 metric_type="IP",
```

```

 consistency_level="Strong"
)

 print(f"Created collection: {collection_name} with dimension_
↪{embedding_dim}")

```

```

[21]: # Set up the collection
 setup_milvus_collection(milvus_client, COLLECTION_NAME, EMBEDDING_DIM)

```

Dropped collection hf\_documentation

Created collection: hf\_documentation with dimension 384

```

[24]: # =====
 # TODO: IMPLEMENT DATA INSERTION (10 points)
 # =====

 def insert_data_to_milvus(
 client: MilvusClient,
 collection_name: str,
 chunks: List[Dict],
 embeddings: List[List[float]],
 batch_size: int = 100
):
 """
 Insert document chunks and embeddings into Milvus.

 Args:
 client: MilvusClient instance
 collection_name: Name of the collection
 chunks: List of chunk dictionaries with text and metadata
 embeddings: List of embedding vectors
 batch_size: Number of records to insert at once

 Returns:
 Total number of inserted records
 """
 total_inserted = 0

 data = []
 for i in range(len(chunks)):
 record = {
 "id": chunks[i]['chunk_id'],
 "vector" : embeddings[i],
 "text": chunks[i]['text'],
 "source": chunks[i]['source']
 }
 data.append(record)

```

```

if not data:
 print("Error: The 'data' list is empty. Check your 'chunks' input.")
 return 0

for i in range(0, len(data), batch_size):
 batch = data[i : i + batch_size]

 # client.insert returns a dictionary or a MutationResult
 result = client.insert(
 collection_name=collection_name,
 data=batch
)
 total_inserted += result.get("insert_count", 0)

return total_inserted

```

```

[25]: # Insert data into Milvus
inserted_count = insert_data_to_milvus(milvus_client, COLLECTION_NAME, chunks,
 ↪ embeddings)

print(f"\nInserted {inserted_count} records into Milvus")

if inserted_count == len(chunks):
 print(" All chunks inserted successfully!")
else:
 print(" Not all chunks were inserted. Check your implementation.")

```

Inserted 5535 records into Milvus  
All chunks inserted successfully!

## 1.7 6. Retrieval

Implement semantic search to retrieve relevant documents for a query.

### 1.7.1 Your Task

Implement the `retrieve_documents` function that: 1. Generates an embedding for the query 2. Searches Milvus for similar vectors 3. Returns the top-K most relevant documents

### 1.7.2 Hints

- Use `embedding_model.encode()` to embed the query
- Use `client.search()` to find similar vectors
- Extract text and source from the search results

```

[26]: # =====
TODO: IMPLEMENT RETRIEVAL (25 points)
=====

```

```

def retrieve_documents(
 query: str,
 client: MilvusClient,
 collection_name: str,
 embedding_model: SentenceTransformer,
 top_k: int = 5
) -> List[Dict]:
 """
 Retrieve the most relevant documents for a query.

 Args:
 query: The search query
 client: MilvusClient instance
 collection_name: Name of the collection to search
 embedding_model: Model to generate query embedding
 top_k: Number of results to return

 Returns:
 List of dictionaries with 'text', 'source', and 'score' keys
 """
 query_embedding = embedding_model.encode([query],
normalize_embeddings=True).tolist()[0]
 search_results = client.search(
 collection_name=collection_name,
 data=[query_embedding],
 limit=top_k,
 search_params={"metric_type": "IP", "params": {}},
 output_fields=["text", "source"]
)

 retrieved_docs = []

 for result in search_results[0]:
 retrieved_docs.append({
 "text": result["entity"]["text"],
 "source": result["entity"]["source"],
 "score": result["distance"] # For IP metric, this is the
similarity score
 })

 return retrieved_docs

```

```

[27]: # Test retrieval
test_query = "How do I fine-tune a transformer model?"

retrieved = retrieve_documents(

```

```

 query=test_query,
 client=milvus_client,
 collection_name=COLLECTION_NAME,
 embedding_model=embedding_model,
 top_k=3
)

print(f"Query: {test_query}")
print(f"\nRetrieved {len(retrieved)} documents:")
for i, doc in enumerate(retrieved):
 print(f"\n--- Document {i+1} (Score: {doc.get('score', 'N/A')}) ---")
 print(f"Source: {doc.get('source', 'N/A')}")
 print(f"Text: {doc.get('text', 'N/A')[:300]}...")

if len(retrieved) == 3 and all('text' in d for d in retrieved):
 print("\n Retrieval test passed!")
else:
 print("\n Check your retrieval implementation")

```

Query: How do I fine-tune a transformer model?

Retrieved 2 documents:

```

--- Document 1 (Score: 0.7484297752380371) ---
Source: huggingface/blog/blob/main/ray-rag.md
Text: ects/rag/finetune_rag_ray.sh) for faster distributed fine-tuning, you can
leverage RAG for retrieval-based generation on your own knowledge-intensive
tasks.

```

Also, hyperparameter tuning is another aspect of transformer fine tuning and can have [huge impacts on accuracy](<https://medium.com/distribut...>)

```

--- Document 2 (Score: 0.7302744388580322) ---
Source: huggingface/blog/blob/main/lewis-tunstall-interview.md
Text: n try to integrate it into your application.

```

So what I've been working on for the last few months on the transformers library is providing the functionality to export these models into a format that lets you run them much more efficiently using tools that we have at Hugging Face, but also just gen...

Check your retrieval implementation

## 1.8 7. Generation

Generate answers using Microsoft Phi-3-mini-4k-instruct/Qwen.

### 1.8.1 Your Task

Implement the `generate_answer` function that: 1. Combines retrieved documents into a context string 2. Formats the prompt using the provided template 3. Generates an answer using the language model 4. Returns a structured result dictionary

### 1.8.2 Hints

- Join document texts with newlines to create context
- Use the `PROMPT_TEMPLATE.format()` to fill in context and question
- Call the generator pipeline with appropriate parameters

1.8.3 <https://huggingface.co/microsoft/Phi-3-mini-4k-instruct>

1.8.4 <https://huggingface.co/microsoft/Phi-3.5-mini-instruct>

1.8.5 <https://huggingface.co/Qwen/Qwen2-1.5B-Instruct>

[ ]:

### 1.8.6 FEEL FREE TO USE A PROPRIETARY MODEL LIKE OPENAI, CLAUDE

```
[36]: import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline

LLM_MODEL = "Qwen/Qwen2.5-1.5B-Instruct"

1. Force MPS (Apple GPU)
device = "mps" if torch.backends.mps.is_available() else "cpu"
print(f"Using device: {device.upper()}")

2. Use float16 to save RAM (8GB limit)
torch_dtype = torch.float16

tokenizer = AutoTokenizer.from_pretrained(LLM_MODEL)
model = AutoModelForCausalLM.from_pretrained(
 LLM_MODEL,
 torch_dtype=torch_dtype,
 device_map={"": device}, # Explicitly map to MPS
 trust_remote_code=True
)

generator = pipeline(
 "text-generation",
 model=model,
 tokenizer=tokenizer
)

print("Model loaded! Generating response...")
```

Using device: MPS

Device set to use mps

Model loaded! Generating response...

### 1.8.7 MODIFY THIS TO SUIT YOUR MODEL

```
[29]: PROMPT_TEMPLATE = """<|system|>
You are a factual research assistant. Your task is to provide a concise answer_
↳based ONLY on the provided context.
Rules:
1. Use ONLY the information in the <context> tags.
2. If the answer is not present, state exactly: "I don't have enough_
↳information to answer this question."
3. Do not use outside knowledge or "hallucinate" facts.
4. Keep the answer under 3 sentences unless requested otherwise.
<|end|>
<|user|>
<context>
{context}
</context>

<question>
{question}
</question>
<|end|>
<|assistant|>
"""
```

```
[30]: # =====
TODO: IMPLEMENT GENERATION (25 points)
=====

def generate_answer(
 query: str,
 retrieved_docs: List[Dict],
 generator: pipeline,
 max_new_tokens: int = 256
) -> Dict:
 """
 Generate an answer using retrieved documents as context.

 Args:
 query: The user's question
 retrieved_docs: List of retrieved document dictionaries
 generator: HuggingFace text generation pipeline
 max_new_tokens: Maximum tokens to generate
 """
```

```

Returns:
 Dictionary with 'answer', 'context', 'query', and 'retrieved_docs'
 """

context = "\n\n".join([doc["text"] for doc in retrieved_docs])

prompt = PROMPT_TEMPLATE.format(context=context, question=query)

outputs = generator(
 prompt,
 max_new_tokens=max_new_tokens,
 do_sample=True,
 temperature=0.7,
 top_p=0.9,
 return_full_text=False # Crucial: only returns the NEW text, not the
↪prompt
)

answer = outputs[0]["generated_text"].strip()

return {
 "query": query,
 "answer": answer,
 "context": context,
 "retrieved_docs": retrieved_docs
}

```

```

[31]: # Test generation
test_query = "How do I fine-tune a transformer model?"

Retrieve relevant documents
retrieved = retrieve_documents(
 query=test_query,
 client=milvus_client,
 collection_name=COLLECTION_NAME,
 embedding_model=embedding_model,
 top_k=3
)

Generate answer
result = generate_answer(
 query=test_query,
 retrieved_docs=retrieved,
 generator=generator
)

```

```

print(f"Question: {result['query']}")
print(f"\nAnswer: {result['answer']}")

if result['answer'] and len(result['answer']) > 10:
 print("\n Generation test passed!")
else:
 print("\n Check your generation implementation")

```

Question: How do I fine-tune a transformer model?

Answer: To fine-tune a transformer model, you can follow these steps:

1. **\*\*Prepare Data\*\***: Collect and preprocess your dataset to be suitable for training a transformer model. This involves tokenizing text data and preparing batches for training.
2. **\*\*Define Hyperparameters\*\***: Choose appropriate values for hyperparameters such as learning rate, batch size, number of epochs, etc., which affect the model's performance.
3. **\*\*Fine-Tune Model\*\***: Use libraries like Hugging Face Transformers or Ray Tune to perform hyperparameter tuning. These tools help optimize the model parameters to achieve better performance.
4. **\*\*Export Model\*\***: Once the best hyperparameters are found, export the trained model in a lightweight format like ONNX (Open Neural Network Exchange). This allows you to run the model efficiently without loading the entire large model file each time.
5. **\*\*Deploy\*\***: Finally, deploy the lightweight model onto servers or cloud services where it can handle inference tasks. Make sure to parallelize the processing to speed up the execution on multiple nodes.

Generation test passed!

[32]: *# Complete RAG pipeline function (DO NOT MODIFY)*

```

def rag_query(
 query: str,
 client: MilvusClient,
 collection_name: str,
 embedding_model: SentenceTransformer,
 generator: pipeline,
 top_k: int = 5,
 max_new_tokens: int = 256
) -> Dict:
 """

```

```

Complete RAG pipeline: retrieve then generate.
"""
Retrieve
retrieved_docs = retrieve_documents(
 query=query,
 client=client,
 collection_name=collection_name,
 embedding_model=embedding_model,
 top_k=top_k
)

Generate
result = generate_answer(
 query=query,
 retrieved_docs=retrieved_docs,
 generator=generator,
 max_new_tokens=max_new_tokens
)

return result

```

```

[33]: # Test complete pipeline with multiple queries
test_queries = [
 "What is the Trainer class in transformers?",
 "How do I load a dataset from HuggingFace?",
 "What is Gradio used for?"
]

for query in test_queries:
 print(f"\n{'='*60}")
 result = rag_query(
 query=query,
 client=milvus_client,
 collection_name=COLLECTION_NAME,
 embedding_model=embedding_model,
 generator=generator,
 top_k=3
)
 print(f"Q: {result['query']}")
 print(f"A: {result['answer']}")

```

=====

Q: What is the Trainer class in transformers?

A: The Trainer class in transformers is used to facilitate the training process by encapsulating various components such as the model, training arguments, dataset objects (for both training and evaluation), and custom metrics. It simplifies the setup and execution of training loops, allowing users to focus on

hyperparameter tuning rather than manual management of resources like GPUs. The Trainer handles tasks from data loading to validation and logging, making it an efficient tool for quickly adapting models to new datasets without needing deep expertise in neural network architectures. It supports different backends including PyTorch and TensorFlow, ensuring flexibility in choosing the best fit for specific projects. Additionally, the Trainer provides utilities for evaluating performance and saving trained models to the Hugging Face Hub, which is crucial for sharing and reproducibility. Thus, the Trainer serves as a streamlined solution for leveraging pre-trained models across diverse applications while maintaining ease of use and scalability.

=====

Q: How do I load a dataset from HuggingFace?

A: To load a dataset from HuggingFace, you can use the `load_dataset` function from the `datasets` library. You provide the name of the dataset on the Hugging Face Hub as the argument, along with the necessary details such as the path to the files if they're stored locally. For example:

```
```python
from datasets import load_dataset

# Load a specific dataset
dataset = load_dataset('glue:mrpc')

# Accessing individual splits
train_data = dataset['train']
validation_data = dataset['validation']

# Accessing elements by their index
first_example = train_data[0]
```
```

This allows you to easily manage and manipulate datasets without needing to host them yourself.

=====

Q: What is Gradio used for?

A: Gradio is primarily used for creating interactive visual interfaces for machine learning models and applications. It simplifies the process of deploying ML tools by allowing users to create web-based applications without needing extensive coding skills. The platform supports various components such as sliders, buttons, graphs, and more, enabling developers to build intuitive user experiences. Additionally, Gradio can handle complex operations like image processing and model loading efficiently, making it suitable for both beginners and experienced users looking to experiment with different machine learning techniques through a simple interface. However, it's worth noting that while Gradio provides a streamlined approach to ML deployment, users may still need basic programming knowledge to fully understand and customize its features.